Automated TEE Adaptation With LLMs: Identifying, Transforming, and Porting Sensitive Functions in Programs

Ruidong Han, Zhou Yang, *Member, IEEE*, Chengyan Ma, Ye Liu, Yuqing Niu, Siqi Ma, *Member, IEEE*, Debin Gao, *Member, IEEE*, and David Lo, *Fellow, IEEE*

***Abstract*—Trusted Execution Environments (TEEs) isolate a special space within a device's memory that is not accessible to the normal world (also known as the untrusted environment), even when the device is compromised. Therefore, developers can utilize TEEs to provide robust security guarantees for their programs, protecting sensitive operations, such as encrypted data storage, fingerprint verification, and remote attestation, from software-based attacks. Despite the robust protections offered by TEEs, adapting existing programs to leverage such security guarantees is challenging, often requiring extensive domain knowledge and manual intervention, which makes TEEs less accessible to developers. This motivates us to design AUTOTEE, the first Large Language Model (LLM) enabled approach that can automatically identify, transform, and port functions containing sensitive operations into TEEs with minimal developer intervention. By manually reviewing 68 repositories, we constructed a benchmark dataset consisting of 385 sensitive functions eligible for transformation, on which AUTOTEE achieves an average F1 score of 0.94 on Java and 0.87 on Python. AUTOTEE effectively transforms these sensitive functions into TEE-compatible versions, achieving success rates of 91.8% and 84.3% for Java and Python, respectively, when using GPT-4o.**

***Index Terms*—Trusted execution environment, LLM-assisted code porting, sensitive-function detection.**

I. INTRODUCTION

TRUSTED Execution Environments (TEEs) provide robust security guarantees for software programs. TEEs create an isolated memory space that remains inaccessible to the normal world (i.e., the untrusted environment), even under a compromised operating system or hypervisor. By executing sensitive operations (e.g., encrypted data storage, fingerprint verification, and remote attestation) within the TEE, the security and integrity of these operations can be guaranteed. This minimizes potential security issues such as data leakage [1], [2], [3], unauthorized access [4], [5], [6], [7], [8], code tampering [9], [10], and software-based attacks [11], [12]. Providing TEE support has become a standard practice for various hardware platforms across different architectures (e.g., Intel [13], [14], AMD [15], ARM [16]).

Unfortunately, despite the advantages offered by TEEs and the wide support provided by mainstream platforms, adapting existing programs to leverage TEE protections is challenging. This process requires extensive domain knowledge and manual intervention for two primary reasons: (i) Security-critical code isolation requirements. To minimize the trusted computing base and reduce the attack surface, developers should isolate only the security-critical code (e.g., cryptographic operations) into the TEE. However, identifying these fine-grained functions requires deep expertise in both application logic and TEE security models [1], [17]. (ii) The constrained execution environment of programs. Most TEEs are designed to run low-level code (e.g., C or Rust) compiled directly to machine instructions, rather than high-level managed languages (e.g., Java or Python) that require dedicated runtime environments. The overhead of securely supporting managed runtimes within the constrained environment of a TEE makes it challenging to adapt these high-level languages. Additionally, even for low-level code, the APIs and system calls available in TEEs may differ from those in the normal world (e.g., Intel SGX does not support timers, file I/O, and multiple threads), necessitating code modifications for correct execution within TEEs.

Several methods have been proposed to facilitate TEE adoption [1], [17], [18], [19]. One branch of research focuses on porting runtime environments for high-level languages to TEEs [18], [19]. However, these approaches introduce significant overhead and may not be applicable to TEEs with small memory sizes. The security guarantees of ported environments may not be as robust as those for native code execution, potentially exposing vulnerabilities such as memory leaks [20], [21]. Another research direction involves separating programs into

Received 29 September 2025; revised 24 December 2025; accepted 15 January 2026. Date of publication 21 January 2026; date of current version 17 March 2026. This research was supported by the National Research Foundation, Singapore, and the Cyber Security Agency of Singapore under its National Cybersecurity R&D Programme (Proposal ID: NCR25-DeSCEmT-SMU). Recommended for acceptance by C. McMillan. *(Corresponding author: Zhou Yang.)*

Ruidong Han, Chengyan Ma, Ye Liu, Yuqing Niu, Debin Gao, and David Lo are with the School of Computing and Information Systems, Singapore Management University, Singapore 188065 (e-mail: rdhan@smu.edu.sg; chengyanma@smu.edu.sg; yeliu@smu.edu.sg; yuqingniu@smu.edu.sg; dbgao@smu.edu.sg; davidlo@smu.edu.sg).

Zhou Yang is with the School of Computing Science, University of Alberta, Edmonton T6G 2N8, Canada (e-mail: zy25@ualberta.ca).

Siqi Ma is with the School of Computing and Information Technology, University of Wollongong, Wollongong 2522, Australia (e-mail: siqim@uow.edu.au).

Digital Object Identifier 10.1109/TSE.2026.3655766

0098-5589 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

sensitive and non-sensitive parts, and only porting the sensitive parts to TEEs [1], [17]. However, existing methods require developers to manually specify data for tracing and manually transform the program, which are complex and error-prone.

Therefore, the current practice of porting programs to TEEs is mainly a human-in-the-loop process that requires substantial manual effort [22], [23], [24], motivating us to propose AUTOTEE, an LLM-enabled approach that can automatically identify, partition, and transform sensitive functions in programs, and port the transformed code to TEEs.

Specifically, for a given program (in Java or Python), AUTOTEE generates the Abstract Syntax Tree (AST) and extracts the leaf functions that do not invoke any other user-defined functions. These leaf functions represent the granular units of computation within the program, which simplifies the porting process and minimizes overhead. Then, AUTOTEE prompts an LLM to review each extracted function and identify those involving sensitive operations (i.e., the use of cryptography and serialization in our paper). Following this, AUTOTEE uses LLMs to automatically transform the identified functions into their functionally equivalent native code implementations. A successful transformation requires that the transformed code (i) compiles successfully (compilable) and (ii) preserves functional equivalence with the original code. For functional equivalence, AUTOTEE prompts the LLM to generate test inputs and employ coverage analysis tools to evaluate both line and branch coverage. AUTOTEE then provides this feedback to the LLM, enabling it to iteratively refine and improve the quality of the generated test inputs to achieve higher coverage. Then, during the validation process, AUTOTEE compares the outputs of the original function and the transformed code under the same test inputs to determine whether they remain consistent.

These steps collectively ensure that the transformed code meets both compilability and functional equivalence requirements. To implement this goal, AUTOTEE adopts the `ReAct` [25] strategy to refine native code implementations, which is an iterative approach that enables LLMs to modify the generated code, validate compilability, and validate functional equivalence through external calls. After the refinement process, AUTOTEE integrates the two components through a port-based communication mechanism that includes attestation capabilities. Specifically, for the original function, AUTOTEE replaces its implementation with a remote call to the corresponding TEE-verified version. Finally, the source program is divided into two parts: one that runs in the normal world and another that executes within the TEE.

A benchmarking dataset was manually constructed from 68 repositories in Python and Java, comprising 385 sensitive leaf functions. Based on this benchmark, experimental results show that AutoTEE achieves an average F1-score of 0.94 on the task of sensitive function detection. We also evaluate the success rate of code transformation. A transformation is considered successful if, for a given set of test inputs automatically generated by an LLM, the transformed code produces outputs identical to those of the original code. In experiments, when applying GPT-4o for transformation, 91.8% of Java code and 84.3% of Python code are successfully transformed.

Contributions:

- We propose an automated approach, AUTOTEE, that re-engineers programs for better adoption to TEEs, minimizing manual code modifications and ensuring support for high-level languages such as Java and Python. It identifies sensitive functions within a program and transforms them into TEE-protected versions.
- AUTOTEE incorporates a robust iterative transformation and validation mechanism, utilizing compiler feedback and functional equivalence checks to guarantee the executability and functional equivalence of the TEE-adapted code.
- We design and implement a mechanism for seamlessly integrating TEE-adapted sensitive functions into existing high-level language programs, including secure attestation and communication channels.

II. BACKGROUND AND MOTIVATION

A. Trusted Execution Environments

Software programs face various security risks, such as unauthorized access [4], [5], [6], [7], [8] and data leakage [2], [3]. To provide stronger protection against such risks, hardware platforms and operating systems offer Trusted Execution Environments (TEEs), which are special areas in the device memory. Logically, the memory in a device is separated into the TEE part and the normal world part. Any computation within the TEE part is not accessible by the normal world, and thus the TEEs naturally provide security protection over many sensitive operations like algorithms, encryption keys, passwords, and personal identification information. Developers are encouraged to run their programs involving such sensitive operations within the TEEs to minimize potential risks. For instance, starting from version 6.0, the Android OS stores the encrypted biometric information (like fingerprints) of users and conducts the verification process within TEEs. The normal world can only observe the verification result, lacking access to the actual verification process. Thus, placing such sensitive operations inside TEEs can enhance the security and integrity of programs. Currently, TEEs have become an essential part of various hardware platforms with different architectures, including Intel SGX [13], Intel TDX [14], AMD SEV [15], ARM TrustZone [16], and OpenTEE [26].

B. Large Language Models & ReAct Prompting

Recent developments in natural language processing (NLP) have led to significant advancements in large language models (LLMs). These models have demonstrated remarkable improvements and have found extensive applications across various fields such as code generation [27], [28], document summarization [29], [30], and security analysis [31], [32]. LLMs have the capability to analyze the context of a given text and make decisions based on the requirements of the task.

However, complex tasks that require logical reasoning abilities, such as code generation, bug repair, and code transformation, continue to pose challenges for LLMs. These tasks

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

necessitate that LLMs decompose the complex task into simpler sub-tasks, complete these sub-tasks by providing reasoning, and subsequently make a final decision. For instance, in code transformation tasks, LLMs must first understand the original code, then transform it into a new form, and finally validate the transformed code to ensure its equivalence with the original code. However, LLMs primarily rely on static internal knowledge, which limits their ability to dynamically assess the quality or correctness of their transformation output through real-time interaction with external environments (e.g., compilers or runtime execution).

`ReAct` [25] is a prompting strategy that enhances LLMs' reasoning capabilities by integrating external tools and actions. It enables LLMs to engage in a "thought-action-observation" loop, interacting with external tools (e.g., databases, computation engines, search engines) to acquire additional information and dynamically validate their decisions. It decomposes tasks into smaller sub-tasks and adapts various actions to acquire information relevant to or to complete these sub-tasks. Therefore, LLMs can review their responses to the task and make appropriate adjustments accordingly. Taking code transformation as an example, compiler messages can help LLMs understand whether the transformed code can be compiled, a basic requirement for successful transformation.

C. Sensitive Operations

In this paper, we define sensitive operations as abstract actions that directly affect program integrity and security. Code functions that implement or directly involve these operations are classified as sensitive functions. Specifically, we focus on cryptographic operations and serialization, following prior TEE-related research [24], [33]. These two categories are detailed as follows:

Cryptographic operation. The purpose of cryptographic operations is to ensure the security, confidentiality, and integrity of critical data. Previous research indicates that cryptographic operations also pose risks of leakage [34], [35] and potential attacks [36], [37]. Protective methods [38], [39], [40] reveal that if these operations are conducted within TEEs, such vulnerabilities can be effectively mitigated. Consequently, functions that perform cryptographic operations, such as encryption, decryption, signature, verification, hashing, and secure key management (e.g., seed generation and random number generation for cryptographic purposes), are classified as sensitive functions.

Serialization. Serialization also poses security risks, as discussed in several studies [41], [42], [43], [44]. Insecure deserialization of untrusted data, for instance, can lead to critical vulnerabilities such as information leakage and, in extreme cases, remote code execution [45]. To address these vulnerabilities, placing the associated operations within a TEE could serve as an effective mitigation strategy.

While we have selected these two as representative categories, in practice, developers can adjust the prompts according to their specific requirements to identify additional sensitive functions.

III. AUTOTEE

A. Methodology Design

Initially, considering the characteristics of TEEs, including resource constraints and limited support for high-level programming languages, our solution focuses on protecting the parts of the program that involve sensitive operations. Developers often struggle to clearly identify which sections of their programs contain sensitive operations [46], [47]. To address this issue, we leverage large language models (LLMs), which are proficient in code semantic analysis [27], [31], [32], to identify sensitive code sections within the program.

For the identified sensitive code, we transform it into a TEE-compatible native implementation. This transformation process is automated using LLMs, incorporating iterative refinement with compiler checks and functional equivalence validations to ensure compilable and functional equivalence.

Subsequently, the transformed TEE-resident code is securely integrated with the original program, enabling protected execution of sensitive operations while minimizing manual intervention.

B. Overview

Fig. 1 illustrates the overview of our approach. Given a program, AUTOTEE first constructs the Abstract Syntax Tree and then extracts the leaf functions from the source code. It then employs an LLM agent to identify functions containing sensitive operations and generate test inputs. Subsequently, AUTOTEE uses a few transformation examples for the LLM to generate new native code. AUTOTEE then checks whether the transformed code can be successfully compiled and validates its functional equivalence by comparing the outputs of the original and transformed code using the same test inputs. If the transformation passes these checks and validations, it is considered successful. Otherwise, AUTOTEE applies the ReAct strategy [25] to iteratively refine the generated code until it satisfies the required criteria (i.e., checks and validations). Once validated, the transformed code is compiled into an executable file and placed within the TEE. Finally, AUTOTEE modifies the original program by replacing the content of the sensitive function with port-based connections that invoke the corresponding function inside the TEE.

C. Sensitive Function Identifier

This module extracts functions from the program and identifies those that are sensitive for further transformation.

1) Leaf Function: A function may involve different dependencies and additional calls. As outlined in Section II, TEEs operate within constrained resource environments. Increasing the scale and complexity of code executed within a TEE can introduce performance bottlenecks and security vulnerabilities, as noted by Liu et al. [48]. Therefore, not all types of sensitive functions are suitable for porting to TEEs. AUTOTEE focuses on leaf functions, which possess the following properties:

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

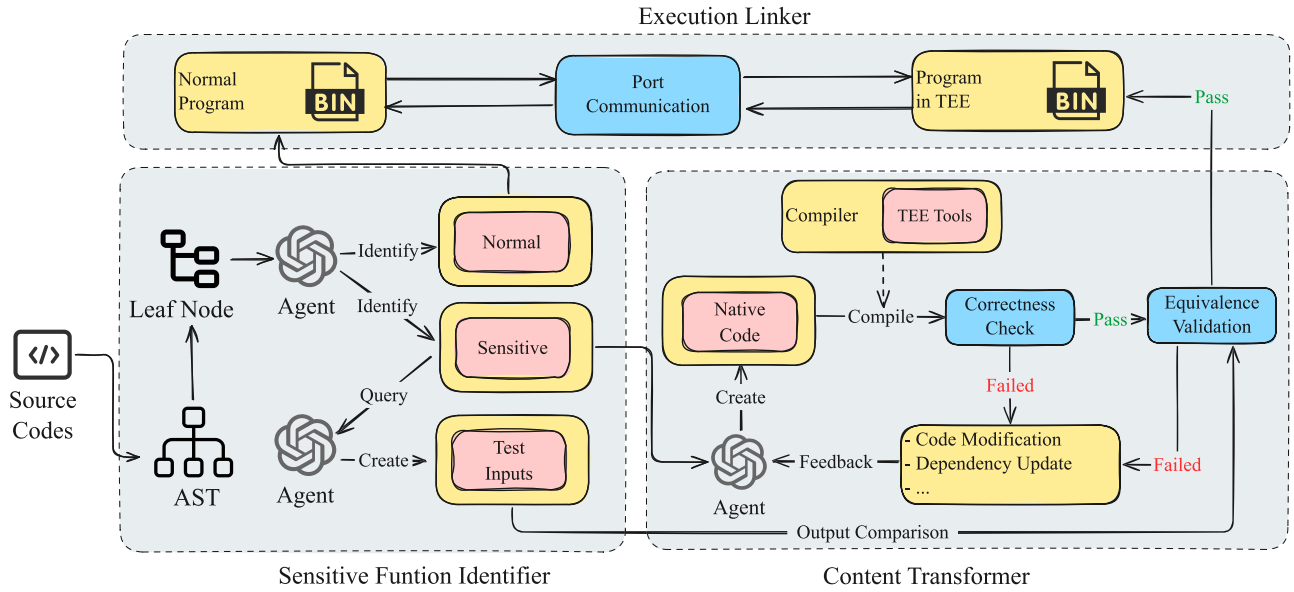


Fig. 1. Workflow and three modules of AUTOTEE.

TABLE I
CROSS-LANGUAGE FUNCTION EQUIVALENCE MAPPINGS

Language	SHA-256 Hashing	Square Root
Python	hashlib.sha256()	math.sqrt(x)
Java	MessageDigest.getInstance("SHA-256")	Math.sqrt(x)
Rust	ring::digest::Context::new(&SHA256)	f64::sqrt(x)

Lowest level of the program structure. These functions reside at the lowest level of the program's structure and do not depend on other user-defined functions. They implement specific sensitive operations and serve as utilities for call chains within the project. Focusing on these self-contained functions avoids the complexity and potential issues associated with entire call chains.

Basic Arguments. The code within the TEE requires interaction with the original application, which is implemented in a different programming language. Cross-language data interaction can introduce various compatibility issues, such as type mismatches and data structure discrepancies. Therefore, we restrict leaf function argument types to primitive data types (e.g., integers, floating-point numbers) and basic composite structures (e.g., arrays and strings) to ensure reliable and secure interaction.

Basic Library Support. AUTOTEE can process leaf functions that invoke public libraries. Unlike calls to developer-defined functions, calls to standard libraries do not require AUTOTEE to understand or analyze their internal implementation. Many commonly used libraries have equivalent implementations across different programming languages, as shown in Table I. During porting, AUTOTEE simply replaces calls to such libraries in the source language with corresponding equivalents in the TEE environment.

AUTOTEE extracts leaf functions through a static analysis process relying on Abstract Syntax Tree (AST) processing. Initially, source files undergo parsing using language-specific AST tools; for instance, the `ast` module is employed for Python, and `JavaParser` for Java. AUTOTEE then traverses the generated ASTs to identify function or method declarations. For each identified declaration, AUTOTEE analyzes the function body to assess whether it invokes other user-defined functions within the project. A function is designated as a leaf function if it does not contain calls to any such functions.

2) Sensitive Function Identification: For the functions, AUTOTEE employs an LLM to identify those that contain sensitive operations at the function level. LLM analyzes the textual content to detect explicit indicators of security-sensitive behavior. Therefore, sensitive functions that AUTOTEE can handle should contain well-known security-related APIs (e.g., `hashlib.sha256`, `cryptography.fernet.Fernet`), sensitive keywords (e.g., encrypt, decrypt, password, token), or standard cryptographic patterns (e.g., AES-based encryption logic). AUTOTEE uses multi-round prompting, asking a series of prompts one by one. Each prompt builds on the previous one, allowing the model to review its responses and correct prior errors. Fig. 2 provides a visual overview of our prompts, with further elaboration below: green {Prompt} represents the prompt; red {Response} represents the response; orange {Type} indicates the specific sensitive type; and cyan {Code} indicates the code.

Prompt 1: Code context. The initial prompt instructs the LLM to check whether a function uses cryptography or serialization operations. Furthermore, it specifies that the response should be either "Yes" or "No" to constrain the LLM's output and facilitate automated processing.

Prompt 2: Sensitive type. The second prompt identifies the type of operation involved. The LLM will return specific types in a list; for example, `[Decryption, Verification]`. Otherwise, the LLM will revise its previous response to "No"

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

- **Prompt 1:** Does this function utilize or implement any operations related to [cryptography, serialization]? Please respond with 'Yes'; otherwise, respond with 'No.' Specifically, cryptography includes [Encryption, ...]; serialization includes [Serialization, ...].
- **Response 1:** Yes or No.

- **Prompt 2:** What type of operation does this function involve?
- **Response 2:** [Encryption, Verification, ...].
- **Or:** Sorry, the previous response is incorrect.

- **Prompt 3:** List the statements involved in [Encryption, Verification, ...].
- **Response 3:** [Encryption: [code1,...], Verification: [code2, code3], ...]
- **Or:** Sorry, the previous response is incorrect.

Fig. 2. Prompts for sensitive function identification.

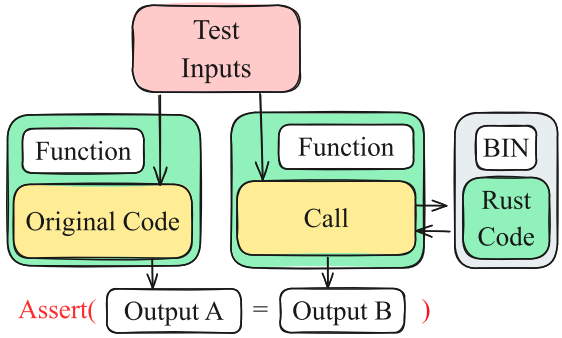


Fig. 3. Process of validation. The term BIN refers to the executable file.

upon realizing that the function does not contain the corresponding operation.

Prompt 3: Statements. The final prompt requests the LLM to enumerate the statements that provide evidence for sensitive judgments. Similarly, the LLM may identify and correct any earlier errors in its analysis.

If, after these three prompts, the LLM still determines that the function contains sensitive operations, AUTOTEE labels the function as "sensitive"; otherwise, it is labeled as "non-sensitive".

3) Test Inputs Generation: Similar to previous approaches in code translation that use test cases to evaluate correctness at the function level [49], [50], [51], AUTOTEE employs test inputs for functional equivalence validation. After identifying sensitive functions, AUTOTEE instructs LLMs to generate test inputs for them. Inspired by recent advances in LLM-based testing [52], [53], AUTOTEE adopts an iterative, coverage-guided generation strategy using both line and branch coverage. The process begins with an initial prompt that explicitly guides the LLM:

> "*Generate a set of diverse test inputs that invoke this function with the goal of maximizing both line and branch coverage.*"

For each generated test suite, AUTOTEE executes the code and measures coverage using `JaCoCo` (for Java) and `pytest-cov` (for Python). If full coverage is not achieved, the resulting coverage report is fed back to the LLM to guide the generation of additional test inputs. This feedback loop continues until either 100% line and branch coverage is achieved or no improvement is observed over three consecutive iterations, a stopping criterion that reflects practical limits due to LLM capability saturation or granularity in coverage reporting.

D. Code Transformer

For the identified sensitive functions, AUTOTEE transforms them into native code. To mitigate potential vulnerabilities within the TEE, such as buffer overflows, dangling pointers, and uninitialized memory, we consider Rust, a *memory-safe* native programming language [54], [55], as the target for transformation.

1) Initial Transformation: Initially, AUTOTEE prompts the LLM to analyze the code and utilizes few-shot learning techniques [56] to produce an initial transformation.

Prompt 1: What objective does this function accomplish? This initial question prompts the LLM to understand the functionality of the code.

Prompt 2: Transform this code into Rust. For example, {source code} to {rust code} with dependency [A, B]. AUTOTEE provides the LLM with three transformation examples, manually generated by ourselves, which the LLM can use as a reference to generate the initial transformation.

2) Compilability Check and Equivalence Validation: AUTOTEE first checks whether the initially transformed code is compilable (i.e., no compilation errors). It uses `Cargo` (the Rust compiler) to compile the code. If no issues are detected, AUTOTEE proceeds to validate the functional equivalence of the transformed code. Specifically, Fig. 3 shows the process of this equivalence validation. AUTOTEE compiles the transformed code into an executable file. Subsequently, AUTOTEE modifies the original code function with an external call pointing to the Rust executable file. AUTOTEE then executes all test inputs to verify whether the outputs of the original and transformed code are identical. Cryptographic operations like seed generation and key derivation inherently introduce randomness, often leading to inconsistent outputs even between functionally equivalent implementations. To establish determinism for testing or validation, we either mock the random source with predefined values or, when supported by the API, use a fixed seed.

If the code fails to compile or if the equivalence validation does not succeed, AUTOTEE carries out subsequent `ReAct` actions to iteratively refine the code.

3) Iterative Refinement: This step requires the LLM to modify the code to ensure that it successfully passes both the compilability check and the equivalence validation. AUTOTEE uses the `ReAct` strategy, which is an iterative process that involves the LLM taking actions to refine the code. Fig. 4 illustrates example prompts utilized by `ReAct`, where the blue Thought/Action represents the thought process of `ReAct`. Following the ReAct format, each response from the LLM will contain its *Thought* along with the corresponding *Action*. The LLM will take actions to refine the code, with each action generating feedback referred to as an observation.

The action space defines the capacity of the LLM agent to refine the code. AUTOTEE provides the following actions:

Action 1: Issues Search. The compiler's check process also generates an error code for specific issues (e.g., "Rust E0308"). The Rust official documentation includes itemizes of the

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

- **Task Prompt:** Implement this code using Rust with the same input type and return type {code}.
- **Thought 1:** We need to ensure that all necessary dependencies are included in ...
- **Action 1:** Dependency update ([md-5, rand,...]).
- **Observation 1:** Warning: use of deprecated function 'base64::encode': Use Engine::encode...

- **Thought 2:** The errors indicate issues with unused imports, missing methods, and ...; I need to address these issues by removing the unused imports ...
- **Action 2:** Code modification ({rust code}).
- **Observation 2:** Error: no method named 'xxx' found for struct 'XXX' in the current scope...

... N time iterations ...

- **Observation N:** Compilation successful; test input outputs are equivalent. Obtain results and exit the task.

Fig. 4. ReAct prompting to solve the code modification.

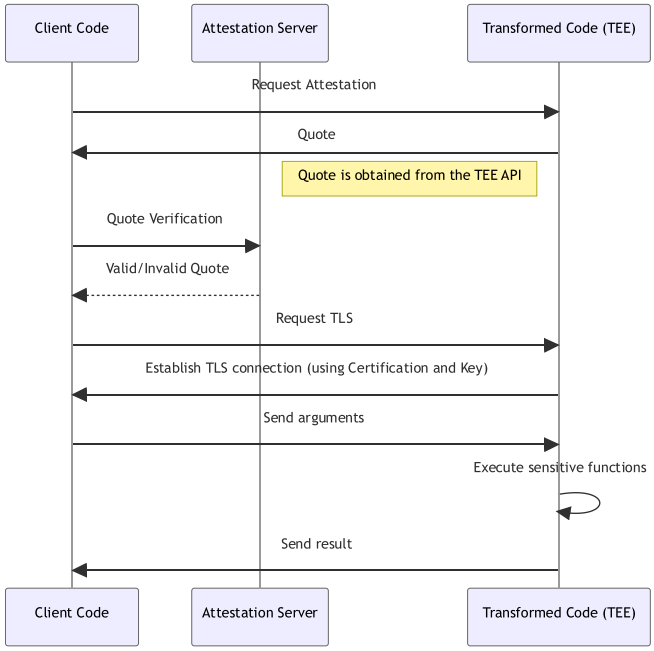


Fig. 5. Secure interaction between source code and TEE code.

index, along with example solutions. The LLM can leverage this action to acquire information concerning the issue and its solutions.

Action 2: Code Modification. This action provides the LLM with an interface to modify the code and write it to the Rust file.

Action 3: Dependency Update. This action adds the corresponding Rust library to the Rust project, such as `use md5;`

Action 4: Compilability Check. This action invokes the compiler (Cargo) to check the code. The compiler will return the check results, indicating issues present within the code, such as the absence of packages and erroneous API calls. This information will serve as guidance on how to modify the code, ensuring that the transformed code is compilable.

Action 5: Equivalence Validation. When the code is compilable (compilability check generates no issues), the LLM will invoke this action to perform the equivalence validation. The process of equivalence validation is equivalent to the itemize provided in Section III-D2. If all outputs are equivalent, the LLM will receive "equivalent" feedback; Otherwise, it will receive "different" feedback along with an indication of which test input is different.

The entire iterative process continues until the program is executable and passes equivalence validation, or until the number of iterations reaches a threshold TH. TH is set to 20 in our evaluation; in actual use, developers need to set it according to the specific LLM's performance characteristics.

4) Platform Adaptation: After several rounds of refinement, AUTOTEE produces a functionally equivalent and TEE-executable code version. Current TEE implementations are broadly categorized into two types: VM-based (e.g., AMD SEV and Intel TDX) and process-based (e.g., Intel SGX, OpenTEE, and ARM TrustZone). The VM-based approach typically maintains compatibility with standard libraries, whereas the process-based approach often requires adaptations to address compatibility issues. As a result, for process-based TEEs, AUTOTEE modifies certain API calls or system interactions accordingly. In TEE environments, programs are launched with an empty environment, meaning no existing environment variables are available. When the generated code relies on environment variables, AUTOTEE reads these values during preprocessing and hardcodes them directly into the source code. Process-based TEEs do not support CPU hardware timers. In such cases, if the code uses CPU timer functionality, AUTOTEE replaces it with a system-level timer API.

E. Execution Linker

When a sensitive function is transformed successfully, AUTOTEE links the original program to its corresponding implementation in Rust. Since these two components are in different languages and operate in distinct environments, i.e., one executing in the normal world and the other within a TEE, AUTOTEE implements their interaction using a secure, port-based communication mechanism.

Both components require code modifications to support a secure interaction process. Fig. 5 illustrates the detailed interaction logic between the client code and the transformed code, which is defined as follows:

Client Code. New code logic replaces the original code, implementing TEE calls.

Transformed Code. A functionally equivalent version of the original code, generated for execution within the TEE.

Attestation Server. An external component used to verify the authenticity and integrity of the transformed code. These servers are typically provided by TEE SDK suites and expose open APIs for verification, such as Intel Attestation Services for SGX and AMD Key Distribution Service for AMD platforms.

The workflow between the client code and the transformed code is as follows:

1) The client code requests the transformed code to perform attestation and return a TEE-generated report (quote).

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

2) The transformed code forwards the quote to the attestation server, which confirms its legitimacy and integrity.
3) Upon successful verification, the client code initiates the establishment of a secure TLS channel with the transformed code.
4) After the TLS connection is established, the client code sends encrypted function arguments to the transformed code, which decrypts and executes them within the TEE.
5) Upon completion, the transformed code sends it back to the client code over the secure TLS connection.

After the corresponding logical modifications are made to the program's code, the transformed code will be compiled into a binary and signed using the TEE SDK.

IV. Evaluation

We evaluate the performance of AutoTEE by answering the following research questions:

- **RQ 1: Identification Accuracy.** Can AutoTEE accurately identify sensitive functions within a program?
- **RQ 2: Transformation.** Can AutoTEE ensure that sensitive functions are correctly transformed, achieving the equivalent functionality as the original code?
- **RQ 3: Resource Consumption.** What is the resource consumption in code transformations?

A. Experiment Preparation

Experiment Environment. We ran AutoTEE on a server equipped with an AMD EPYC 7763 processor and an NVIDIA RTX 6000 Ada graphics card. We chose four LLMs: GPT-4o [57], Qwen2.5:32B [58], DeepSeek-v3 [59], and LLaMA3.1:8B [60] in our experiments, subsequently referred to as GPT-4o, Qwen2.5, DeepSeek, and LLaMA3.1, respectively. Qwen2.5 and LLaMA3.1 are deployed locally on our server. For TEE platforms, we selected Intel SGX (Intel i7-9700 processor). Our evaluation includes both locally deployed models and commercial models accessed through an application programming interface. The objective of this mixed approach is to benchmark a diverse range of models, not to endorse the use of external services for workflows involving sensitive data.

Dataset Construction. To the best of our knowledge, no existing dataset of functions to be protected under TEEs is available. Thus, we manually constructed a benchmark for the task addressed in our paper. To identify repositories that potentially contain sensitive operations, we conducted a keyword search on GitHub, targeting repository names and itemizes containing one or more of these terms: *password, credential, seal, crypto, token, serialization, and cryptography.* After identifying these projects, we cloned the repositories locally. To ensure the quality of collected projects, we only kept projects with more than 50 stars, a commonly used threshold to exclude toy projects [34], [35]. Finally, we obtained 38 Java repositories and 30 Python repositories. By parsing these projects, we obtained 7,214 and 3,770 Java and Python functions, respectively. We retained the leaf functions and conducted a manual review involving three authors with backgrounds in cryptography or software engineering. Our ground truth labels were established by human annotators using the same criteria: (1) it directly performs security-sensitive operations (e.g., encryption, authentication, key generation); (2) it explicitly invokes known cryptographic or deserialization APIs (such as hashlib.sha256 or cryptography.fernet.Fernet); or (3) it contains recognizable cryptographic patterns (e.g., AES-based encryption logic). Only the functions that were classified as sensitive by all three reviewers were kept for further analysis. From examples marked as sensitive, we randomly sampled two cryptography-related functions (e.g., AES encryption, RSA key generation) and one serialization function as examples that we used as "few-shot" learning instances. The remaining sensitive functions were used in the subsequent experiments, including in 232 sensitive functions for Java and 153 for Python. In terms of lines of code (LoC), a commonly used proxy for implementation scale in empirical software engineering research [61], [62], the average LoC is 18.2 for Java functions (range: 3–95) and 15.6 for Python functions (range: 3–83). Cryptographic operations and serialization typically follow established usage patterns [63], [64] (e.g., for encryption: initializing a key, using the key for encryption or decryption, and outputting the result), so the logic tends not to be highly complex and the LoC is generally not very large. Additionally, to construct our dataset, we randomly added 241 Java and 166 Python normal functions from the remaining code (i.e., normal functions).

Few-shot Example. From examples marked as sensitive, we sampled two cryptography-related functions (e.g., AES encryption, RSA key generation) and one serialization function as examples that we use as "few-shot" learning instances. For each chosen function, one author manually implemented a Rust transformation, focusing on preserving semantic equivalence while re-expressing syntactic structures. The resulting transformed examples were then reviewed by the other two authors to evaluate logical equivalence. Any discrepancies in equivalence identified during this review were resolved by manually refining the transformations.

B. RQ1: Identification Accuracy

1) Identification Result: In this experiment, we applied AutoTEE to the dataset to evaluate its effectiveness in sensitive function identification. We repeated each experiment 10 times for each model on our benchmark and reported the average results in Table II. The evaluation metrics include average precision, recall, and F1-score. On both datasets, LLaMA3.1 achieved the lowest F1-scores, with 0.906 on Java and 0.861 on Python, although its precision on Python was higher than that of DeepSeek and Qwen2.5. On the Java dataset, GPT-4o achieves the best performance across all three metrics. On the Python dataset, GPT-4o leads in precision and F1-score, while DeepSeek and Qwen2.5 achieve higher recall scores. Specifically, GPT-4o attains the highest average F1-score, with 0.958 on Java and 0.883 on Python, followed by DeepSeek, with 0.944 on Java and 0.881 on Python, and Qwen2.5, with 0.942 on Java and 0.872 on Python. To assess whether the observed performance differences are statistically significant,

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE ON SENSITIVE FUNCTION DETECTION (AVERAGE)

Model	Java			Python		
	Precision	**Recall**	**F1**	**Precision**	**Recall**	**F1**
GPT-4o	94.8%	96.8%	0.958	88.1%	88.5%	0.883
DeepSeek	92.2%	96.7%	0.944	84.9%	91.5%	0.881
Qwen2.5	94.8%	93.7%	0.942	83.8%	90.8%	0.872
LLaMA3.1	90.6%	90.5%	0.906	85.4%	86.9%	0.861

TABLE III
IDENTIFICATION RESULT WITH DIFFERENT POLICY

Qwen2.5	**Java**			**Python**			**R.T.**
	P	**R**	**F1**	**P**	**R**	**F1**	
P1	87.1%	86.6%	0.868	75.6%	83.1%	0.791	0.3s
P1&2	90.9%	90.9%	0.909	79.0%	86.3%	0.825	3.1s
P1&2&3	93.5%	92.7%	0.931	83.8%	90.9%	0.868	7.7s

* PX denotes Prompt-X; **P** denotes precision; **R** denotes recall; **F1** denotes F1-score; **R.T.** denotes the average response time in seconds.

we applied the *Mann-Whitney U Test* [65] to compare the F1-scores of GPT-4o with those of DeepSeek, Qwen2.5 and LLaMA3.1. For each comparison, the test was conducted on 10 F1-scores obtained from the 10 repeated experimental runs. On the Java dataset, GPT-4o significantly outperforms DeepSeek, Qwen2.5 and LLaMA3.1 at the 99% confidence level ($p < 0.01$). On the Python dataset, GPT-4o performs significantly better than Qwen2.5 and LLaMA3.1 at the 99% confidence level ($p < 0.01$), while the difference compared to DeepSeek is not statistically significant ($p > 0.05$). Overall, GPT-4o demonstrates the strongest performance in sensitive function detection. DeepSeek serves as a viable secondary alternative.

For these failure cases, we conducted a manual examination to analyze their characteristics. Functions that contain multiple shift operations or array manipulations may be incorrectly identified as sensitive operations because bit shifting is a commonly used operation in cryptography for security handling. Additionally, if a function involves operations related to a dictionary with key-value pairs, the term "key" may be improperly identified as referring to cryptographic operations. The performance gap between Java and Python in sensitive code identification can be attributed to inherent differences in language design and coding practices. Specifically, research has shown that statically-typed languages like Java facilitate automated code analysis due to their explicit type usage and structured APIs [66], [67]. In contrast, Python's dynamic typing, flexible syntax, and idiomatic patterns such as runtime attribute access may obscure the intent or presence of sensitive operations, making it more challenging for LLMs to detect these code segments reliably.

2) Prompt Strategy Influence: In addition, we evaluated the impact of three different prompt configurations on the detection performance. Using Qwen2.5 as an example, we conducted another detection under three scenarios: using only prompt 1, a combination of prompts 1 and 2, and the full set including prompts 1, 2, and 3 (the setup used in the previous experiment). The results are summarized in Table III. As shown in the table, the combined prompt configuration achieves higher accuracy compared to using a single prompt. For Java, the F1 score increased from 0.868 to 0.909 (P1&2) and further to 0.931 (P1&2&3). Similarly, Python's F1 improved from 0.791 (P1) to 0.868 (P1&2&3). However, generating longer outputs inherently consumes more time and resources due to the nature of LLMs. Our incremental prompting strategy addresses this by applying increasingly detailed prompts in stages, first filtering likely candidates with a concise prompt (Prompt 1), and then refining the results with additional prompts only when necessary. This staged approach effectively reduces unnecessary token consumption while preserving detection accuracy.

C. RQ2: Transformation

This experiment uses a dataset of 232 Java functions and 153 Python functions, all of which were manually labeled as sensitive. By using all sensitive functions as the input for RQ2, we ensure that the evaluation of AUTOTEE 's transformation effectiveness is not confounded by identification errors from RQ1. For each of these functions, AUTOTEE transforms the code into Rust. We validated the equivalence after transformation, analyzed the reasons for failures, and compared AUTOTEE with other prompting methods.

1) Metric: Before launching the experiment, we established several metrics for measurement.

- **#Sensitive:** Refers to the number of cases marked as sensitive, specifically those that require transformation.
- **#Direct:** Refers to the number of cases that directly achieve executability and pass the equivalence validation after an *initial transformation*, with no need for activating *iterative refinement*.
- **#Equivalent:** Refers to the number of cases for which functional consistency with the original code is confirmed following either an *initial transformation* or *iterative refinement*. This count encompasses cases categorized as #Direct.
- **Avg. Iter.:** Refers to the average number of iterations completed during successful transformations. Cases classified as #Direct are not included in this statistic.

2) Coverage of Generated Test Inputs: The system first generates unit test inputs for these sensitive functions. We leverage `JaCoCo` for Java and `Pytest-cov` for Python to measure line and branch coverage of these test inputs. On average, the Java test inputs achieve 98.2% line coverage and 100% branch coverage, while the Python test inputs achieve 100% for both line and branch coverage. The reason Java does not reach 100% coverage is that when a sensitive function is defined as a static method within a class, our test inputs do not trigger the class's initialization function. However, if we only calculate coverage on sensitive functions, we can achieve 100% coverage.

3) Initial Transformation: Initially, referring to the first step in Section II-C, we applied several prompts and examples to transform the code into Rust. The *#Direct* row in Table IV presents the results. The table shows that, solely with the

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PERFORMANCE OF AUTOTEE FOR TRANSFORMING SENSITIVE FUNCTIONS

Metric	Java (232 Samples)				Python (153 Samples)			
	GPT-4o	DeepSeek	Qwen2.5	LLaMA3.1	GPT-4o	DeepSeek	Qwen2.5	LLaMA3.1
#Direct	48 (20.7%)	43 (18.5%)	41 (17.6%)	0 (–)	29 (18.9%)	31 (20.3%)	20 (13.1%)	0 (–)
#Equivalent	213 (91.8%)	203 (87.5%)	192 (82.7%)	4 (1.7%)	129 (84.3%)	117 (76.5%)	103 (67.3%)	4 (2.6%)
Ave. Iter.	5.3	5.5	6.5	9.5	5.6	5.8	6.8	13.3

provided prompts and examples, the LLM agent is unable to successfully transform the majority of the cases, achieving a success rate of less than 21%. We manually analyzed these *Direct* cases and summarized their characteristics. Typically, they contain only a few simple statements, including direct processing of strings (e.g., hashing) and returning the result without any branching. However, a high failure rate indicates that only providing prompts and examples cannot meet our transformation requirements, as other cases may contain complex operations, such as standard library imports, initialization of cryptographic algorithms, and pertinent error handling.

4) Iterative Refinement: For remaining cases that did not achieve successful transformation, AUTOTEE carried out *iterative refinement* with the `ReAct` strategy. The *#Equivalent* row in Table IV illustrates the results that achieve successful transformation. Meanwhile, the *Ave. Iter.* row represents the average number of iterations required to attain success. Overall, the success rate of GPT-4o is the highest, regardless of the programming language employed. In Java transformation tasks, GPT-4o achieved a 91.8% success rate (213 cases), followed by DeepSeek at 87.5% (203 cases), and Qwen2.5 at 82.7% (192 cases). LLaMA3.1 lagged behind, transforming only 4 cases with a 1.7% success rate. Python transformations require more iterations and yield lower overall success rates. GPT-4o transformed 129 cases with an 84.3% success rate. Qwen2.5 managed a 67.3% success rate on 103 cases, while DeepSeek achieved 76.5% success on 117 cases. LLaMA3.1 again lagged significantly, transforming only 4 cases with a 2.6% success rate.

To assess the statistical differences in performance among GPT-4o, DeepSeek, Qwen2.5, and LLaMA3.1 we conducted pairwise *McNemar's tests* [68]. For a given set of function instances, this test compares the results of two models on the same instances and evaluates whether their disagreements are systematically biased toward one model. Specifically, it examines instances in which one model successfully transforms a function while the other fails, and tests whether such asymmetries are statistically significant. The results show that GPT-4o significantly outperforms DeepSeek, Qwen2.5, and LLaMA3.1 on Java and Python ($p < 0.05$). In addition, DeepSeek significantly outperforms Qwen2.5 and LLaMA3.1 on both Java and Python ($p < 0.05$), indicating that it consistently ranks second and, due to its open-source nature, represents a recommended base LLM for local deployment in privacy-critical scenarios.

5) Category Influence: Additionally, Table V summarizes the success rates of different function categories. The results indicate that, on average, models perform better on cryptographic operations than on serialization tasks. Specifically, cryptographic operations exhibit average model success rates of 87.3% for Java and 84.1% for Python. In contrast, models achieve somewhat lower but still robust average success rates of 78.6% in Java and 74.2% in Python on serialization-related functions. This performance gap is attributed to the nature of the tasks: cryptographic operations often follow standardized patterns and rely on well-documented libraries (e.g., `javax.crypto, hashlib`), making them easier for models to recognize. In contrast, serialization logic tends to involve more heterogeneous structures, nested data handling, and format-specific transformations, which increase semantic complexity.

TABLE V
SUCCESS RATES FOR AGGREGATED FUNCTION TYPES

Type	GPT-4o		DeepSeek		Qwen2.5	
	J	P	J	P	J	P
Crypto	93.0%	86.6%	88.2%	74.1%	82.9%	69.6%
Serial	86.7%	80.0%	84.4%	67.5%	82.2%	62.5%

* **J** indicates Java; **P** indicates Python. Due to very few successful samples, results for **LLaMA3.1** are omitted.

6) Code Language Influence: Part of Python's lower success rate compared to Java is attributed to its highly integrated language design and dynamic typing system. On the one hand, some libraries available in Python offer high-level components that are not directly available in Rust. For example, in one python example, the `Crypto.Util.number` module provides functions such as `bytes_to_long`, `getPrime`, and `inverse`. In contrast, Rust's equivalent types, such as BigUint, only include an implementation of `bytes_to_long`, while functions like getPrime and `inverse` must be generated by the LLM, increasing transformation complexity. On the other hand, as an interpreted language, Python lacks explicit type declarations, which limits the amount of semantic information available to the LLM. Conversely, the target language, Rust, being statically typed, requires more type information for transformation. For verification, we added type annotations for the arguments and return values of failed cases for re-transformation. This process is done semi-automatically: first, we use `Copilot` [69] in VSCode to provide a recommendation; then, we manually assess the recommendation. We manually insert the type annotations and verify that the code still passes the previously generated test inputs. Table VI

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PYTHON TRANSFORMATION RESULTS WITH ADDITIONAL TYPE ANNOTATIONS

	GPT-4o	DeepSeek	Qwen2.5
W/O Ann.	129 (84.3%)	117 (76.5%)	103 (67.3%)
W Ann.	132 (+4, 86.2%)	126 (+9, 82.3%)	113 (+10, 73.9%)

* **W/O Ann.** refers to without type annotation; **W Ann.** refers to with type annotation;

TABLE VII
THE SUCCESS RATE UNDER DIFFERENT FUNCTION SIZES

Language	LoC	GPT-4o	DeepSeek	Qwen2.5
Java	≤ 10	91.4	88.2	82.8
	$11 \sim 25$	91.3	87.5	82.7
	> 25	91.4	85.7	80.0
Python	≤ 9	84.1	76.8	66.7
	$10 \sim 21$	85.2	77.0	67.2
	> 21	82.6	73.9	65.2

illustrates their results. Among these cases, GPT-4o successfully transformed an additional 4, Qwen2.5 transformed 10 cases, and DeepSeek transformed 9 cases. This resulted in success rates of 86.2%, 73.9%, and 82.3%, respectively. Given that GPT-4o already achieved the highest success rate, its improvement is not substantial. The success rate of Qwen2.5 has improved most significantly. DeepSeek had nearly matched the performance of GPT-4o. It is evident that for interpreted languages, the success rate can be enhanced through the manual addition of type annotations.

7) Code Complexity Influence: To investigate how code complexity affects the accuracy of sensitive code identification, we analyzed model performance with respect to the number of LoC in each function, as a practical measure of code complexity. We categorized all functions into tertiles three roughly equal-sized groups) based on their LoC distribution for each programming language. This ensures that our analysis of performance across different function complexities is based on a balanced number of samples in each category. This resulted in specific boundaries: for Java, the three groups correspond to ≤ 10, $11-25$, and > 25 LoC, which contains 77, 78 and,77 samples respectively; for Python, the three groups correspond to ≤ 9, $10 - 21$, > 21 LoC, where each group contains 51 samples.

The results are summarized in Table VII. For both Java and Python, all models achieved stable performance across the categories, with success rate variations within $\pm 2.5\%$. These results indicate that the performance remains more or less stable with the increased complexity.

8) Integration and System-Level Testing: To assess functional equivalence, we leveraged each project's existing unit test suite. The validation process involved replacing the original Java/Python function bodies with proxies that preserved the original signatures but forwarded inputs to the TEE-hosted Rust implementation and returned its outputs. We then executed the full unit test suite for each project to confirm that the number of test failures did not increase compared to the original, unmodified project. We used the transformations generated by GPT-4o for this experiment, as they consistently achieved the highest success rate in our other evaluations. The scope was restricted to projects with unit test suites executable via standard build tools (e.g., Maven, Gradle, pytest), excluding those requiring specialized runtimes like the Android SDK or a GUI. The resulting benchmark comprises 21 Java projects and 6 Python projects, encompassing 1,248 unit tests for Java and 328 for Python, respectively. The Java projects contained 94 of our transformations, while the Python projects included 15 transformations. Our baseline execution of the original tests resulted in 12 errors and 22 skips for Java, and 5 errors and 6 skips for Python. Our validation criterion was that no new tests failed after integrating the transformed Rust code.

The results showed that for Java, two of the 94 transformations led to new test errors. For Python, one of the 15 transformations introduced a failure. Manual inspection revealed that these failures were not caused by logical errors in the transformed code but by environmental incompatibilities with the TEE. The Java failures were due to an attempt to invoke a dynamic link call for an elliptic curve algorithm, which is unavailable in the TEE. The Python failure was caused by reliance on reading a temporary key from a local file, an I/O operation not permitted across the TEE boundary.

9) Failure Analysis: For the code that failed to transform, we manually inspected its implementation and structure to identify key issues, excluding LLaMA3.1 from this analysis due to its extremely low success rate.

Sophisticated cryptography (46.2%). If a code employs sophisticated and multiple algorithms, the LLM may fail to accurately transform the code. For instance, Java code simultaneously utilizes `Curve25519` public keys and `XSalsa20` as a stream cipher for encryption, ultimately employing the `Poly1305` algorithm for authentication. AUTOTEE inaccurately transforms the code due to the complexity of the cryptographic operations, preventing successful equivalence validation before the iteration threshold was reached.

Shift operations (22.9%). Even if the code transformation appears successful, many transformed programs with bit shift operations terminate with execution errors. For these cases, the model translations are wrong. There are two possible reasons: (i) LLMs lack the precise reasoning required for bit-shifting; (ii) there are different semantics of bit shift operations among Java, Python, and Rust. It is hard to determine if only one or both factors influence the wrong translations and which one is the most influential.

Missing flag variable (14.7%). There is a global static flag whose specific value the LLM cannot determine, resulting in continuous incorrect substitution of variables.

Functionality change (9.1%). Among the cases that did not pass equivalence validation, there are instances of "security upgrade" modifications. For example, the original code employed *SHA-1* for hash computation, while the transformed code replaced it with *SHA-256*, thereby enhancing security. Although this results in bias, from a security standpoint, *SHA-256*

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

Original Python code

```
from passlib.hash import ldap_salted_sha1
def hash(a):
    # hash salt
    h = ldap_salted_sha1.hash(a)

    return h
```

Transformed Rust code

```
use sha2::{Sha256, Digest};use hex;
fn hash(a: &str) -> String {
    let mut hasher = Sha256::new();
    hasher.update(a);
    hex::encode(hasher.finalize())
}
```

Fig. 6. Inequivalent functionality caused by security method change.

offers greater reliability than *SHA-1*, given that *SHA-1* has been demonstrated to be more vulnerable to collision attacks [70], [71]. Subjectively, we consider this to be a beneficial modification as it offers a more secure implementation of functionality. Unfortunately, the LLM did not always exhibit such reliability. In the absence of sufficient semantic information, particularly with Python code, the LLM may compromise the security of the transformed code while also producing different outputs. For example, Fig. 6 is the original and transformed Python code, which calculates the hash value using the hashing and salting method (recursive hash). Although the LLM substituted *SHA-1* with *SHA-256*, the omission of the salting mechanism still led to a functional inequivalence.

Others (7.1%). There are no obvious code features; we believe it is a problem with the model's own ability.

The distribution of errors indicates that the model struggles the most with shift operations and sophisticated cryptography. This is mainly because the two tasks require strong capability of mathematical reasoning and bit-level data manipulation, making it a challenge for LLMs to infer the underlying logic from text alone and thus leading to error-prone transformation.

10) Comparison and Analysis: AUTOTEE employs multiple methods to assist the LLM in accomplishing the transformation tasks. This sub-experiment still employs LLMs for transformation but uses different methods for comparison:

- **Zero-Shot:** This method directly prompted the LLM to transform sensitive functions into Rust code without providing any additional information.
- **Few-Shot:** This method prompted the LLM to transform the code using three transformed examples. However, unlike the *initial transformation*, it did not require the LLM to analyze the code first.
- **Compilability Check:** Other processes are consistent with AUTOTEE, except that there is no equivalence validation during the iterative refinement. It only utilized compiler checks as feedback for the LLM agent.

Table VIII presents the comparison results. According to the table, the success rate for the *Zero-Shot* is the lowest due to the minimal information provided. When additional transformation examples were introduced (*Few-Shot*), the success rate improved. Despite this enhancement, a significant number of cases (over 80%) still remained unsuccessful. Following the introduction of the *Compilability Check*, the success rate exhibited a substantial increase. *Compilability Check* resulted in the majority of the transformed code being executable; however, it did not ensure its equivalence to the original code. In comparison, since equivalence validation was introduced, AutoTEE achieved the highest success rate, ensuring both executable and equivalent functionality of transformed code.

11) Stability: Utilizing LLMs as agents introduces an element of randomness, which can lead to different transformation outcomes for the same code. Thus, we also evaluate the output stability of AUTOTEE. To reduce API token costs, we randomly sampled approximately 10% of the cases from the original Java and Python datasets for a stability assessment. The final stability assessment dataset comprised 37 instances (i.e., sensitive functions), of which 23 instances came from the Java dataset and 14 instances from the Python dataset.

We conducted four more rounds of repeated experiments, and the results are presented in Table IX. For these 37 samples, GPT-4o, Qwen2.5, and Deepseek successfully transformed 35, 30, and 33 samples, respectively, in the original experiment. The table indicates that, in most cases, the success rate remained equivalent. However, the results also exhibited minor fluctuations. In *Exp1*, GPT-4o successfully transformed one additional sample, whereas Qwen2.5 in *Exp3* failed to transform one previously successful sample. The additional successful transformation was a data encryption process. The failure instance arose from a functionality change, where the source code used *SHA1*, while the transformed result applied *SHA256*. In general, despite the presence of some fluctuations, AUTOTEE demonstrates stability in its transformation outcomes.

D. RQ3: Resource Consumption

This experiment exhibits the resource consumption of AUTOTEE, including LLM consumption and practical TEE consumption.

1) Resource Consumption in LLMs: This study compares the token consumption of LLMs for successful transformations in our experiments. Table X reports the average token usage and monetary cost across the Identification (Ident.), Test Generation (Test Gen.), and Transformation (Trans.) stages, as well as the average cost per iteration incurred during transformation phases (Per. x #Iter). From the table, we observe the following: (1) The token usage of various models during the sensitive function identification phase is similar (approximately 0.48K). (2) The primary differences in token consumption occur in the transformation phase. LLaMA3.1 requires more iterations in the transformation phases, resulting in higher token usage. The other three models perform better than LLaMA3.1. (3) GPT-4o and DeepSeek generate more detailed reasoning within the ReAct iterative process, leading to higher per-iteration token usage but fewer overall iterations. (4) DeepSeek is cheaper

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
COMPARISON OF CODE TRANSFORMATIONS ACROSS DIFFERENT METHODS

Method	Java (232 #Sensitive Samples)				Python (153 #Sensitive Samples)			
	GPT-4o	DeepSeek	Qwen2.5	LLaMA3.1	GPT-4o	DeepSeek	Qwen2.5	LLaMA3.1
Zero-shot	22 (9.4%)	20 (8.6%)	18 (7.8%)	0 (-)	13 (8.5%)	13 (8.5%)	11 (7.2%)	0 (-)
Few-shot	44 (18.9%)	42 (18.1%)	39 (16.8%)	0 (-)	26 (16.9%)	28 (18.3%)	18 (11.8%)	0 (-)
Compilability check	158 (68.1%)	148 (63.8%)	140 (60.3%)	2 (0.8%)	61 (39.8%)	57 (37.2%)	49 (32.1%)	1 (0.6%)
AUTOTEE	213 (91.8%)	203 (87.5%)	192 (82.7%)	4 (1.7%)	129 (84.3%)	117 (76.5%)	103 (67.3%)	4 (2.6%)

TABLE IX
SUCCESS RATES OF TRANSFORMATIONS IN REPEATED EXPERIMENTS

	Original Exp (37 Samples)	Exp1	Exp2	Exp3	Exp4
GPT-4o	35 Equivalent	36 (+1)	35	35	35
DeepSeek	33 Equivalent	33	33	33	33
Qwen2.5	30 Equivalent	30	30	29 (-1)	30

* **Exp** refers to experiment.

TABLE X
AVERAGE TOKEN USAGE AND COST OF LLMS

Model	Avg. Token Usage (K)			Avg. Cost
	Ident.	Test Gen.	Trans.(Per. x #Iter)	
GPT-4o	0.49K	12.4K	36.72K (6.8 K ×5.4)	$0.398
DeepSeek	0.48K	13.2K	39.1K (6.9 K ×5.6)	$0.024
Qwen2.5	0.48K	13.6K	40.4K (6.1 K ×6.6)	–
LLaMA3.1	0.49K	17.3K	65.9K (5.7 K ×11.4)	–

Ident.: average token usage during the identification stage; **Test Gen.**: average token usage during the test generation stage; **Trans.**: average token usage of the successful transformation cases. Values in parentheses denote average tokens per iteration x average number of iterations. Qwen2.5 and LLaMA3.1 are locally deployed and incur no cost.

than GPT-4o. While its total token consumption is comparable to GPT-4o, its cost is lower ($0.024 vs. $0.398), providing a cheaper choice. Qwen2.5 and LLaMA3.1 are deployed locally, so they incur no API costs.

2) Practical Resource Consumption of Execution: We present several case studies to illustrate the practical resource consumption of code executed within the SGX TEE. We used the `Rust - SGX` [72] tool to compile, sign, and load the transformed code into the TEE. For experimental convenience, we implemented a basic attestation service inside the SGX. This service accepts a quote, verifies whether it was generated by the expected TEE instance, and checks whether the content has been altered.

Table XI compares the resource consumption of the original code with that of the transformed code executed inside the TEE. The table includes the following columns: whether an external library is used (Lib.), the original execution time before transformation (Ori. Cost), the execution time inside the TEE (TEE Cost), and the performance overhead expressed as a multiplicative increase (Increase). Each cost value represents the average of five runs to ensure measurement stability. The

TABLE XI
RUNTIME OVERHEAD OF TRANSFORMED FUNCTIONS EXECUTED INSIDE THE TEE

Case	Lib.	Ori. Cost	TEE Cost	Multiple
MD5 (Java)	✓	28 ms	338 ms	12.1×
SHA1 (Java)	✗	18 ms	253 ms	13.1×
Random (Java)	✓	18 ms	254 ms	14.1×
AES Decryption (Java)	✓	42 ms	242 ms	5.7×
Serialization (Java)	✗	13 ms	253 ms	19.4×
RSA Key Gen. (Python)	✓	212 ms	355 ms	1.6×
SHA256 (Python)	✗	0.2 ms	251 ms	1255×
AES-CFB Enc. (Python)	✓	1.2 ms	172 ms	143×

* **Lib.** indicates whether an external library is used in original code; **Ori. Cost** refers to the original execution time before transformation; **Key Gen.** is short for key generation; **Enc.** is short for encryption.

measurements account for TLS communication overhead, quote verification cost, and function execution within the TEE.

In general, the resource consumption of most code executed inside the TEE increases due to transmission delays and attestation verification. Among the evaluated cases, Python's `SHA256` implementation relies on bitwise shift operations and does not use external libraries, resulting in a relatively low execution time in its original version (0.2 ms) but experiencing a significant increase (251 ms) when executed inside the TEE. Similarly, `AES-CFB Enc.`, which utilizes a C-based library, also has a low original execution time and exhibits an overhead of approximately 143× when ported into the TEE. Conversely, for computationally intensive functions like `RSA Key Gen.`, the original execution time (212 ms) is already substantial. In this case, the fixed TEE overhead constitutes a smaller fraction of the total runtime, resulting in a much lower relative increase of only 1.6×.

V. DISCUSSION

AUTOTEE's transformations follow the standard TEE application migration pattern [17], which separates the application into a trusted part and an untrusted part and establishes a secure communication channel. AUTOTEE preserves integrity and confidentiality through a multi-layered security design. To ensure integrity, AUTOTEE integrates standard remote attestation to verify code authenticity before execution and transforms sensitive functions into memory-safe Rust, effectively mitigating runtime memory corruption vulnerabilities (e.g., buffer overflow). To ensure confidentiality, the *Execution Linker* of AUTOTEE establishes a secure TLS channel for all

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

cross-boundary interactions, mandating that all function arguments are serialized and encrypted during transit to prevent data leakage at the interface level.

AUTOTEE reduces the engineering burden of adopting TEEs by automating phases that normally are done manually. First, in partitioning, unlike manual methods that need labor-intensive dependency analysis to establish trust boundaries, AUTOTEE automatically identifies and protects sensitive functions, which avoids the difficulty of untangling coupled application logic. Second, AUTOTEE replaces the manual, error-prone process of porting high-level code into Rust with an LLM-driven transformation pipeline. This transformation eliminates the need for TEE-specific systems expertise during the application development lifecycle. Finally, for validation, AUTOTEE reduces the high costs of manual testing. By combining the ReAct strategy with automated differential testing, AUTOTEE checks functional equivalence between the original and trusted components, freeing developers from writing and maintaining extensive test suites.

VI. THREATS TO VALIDITY AND LIMITATION

A. Threats to Validity

Internal Validity. In AUTOTEE, we assess the equivalence of code transformations by executing the original and transformed functions on the same set of inputs and comparing their outputs. We acknowledge that the current validation may not capture all semantic aspects. To mitigate this threat, we use LLMs to generate test inputs with high coverage, improving test adequacy by maximizing line and branch coverage. Because AUTOTEE targets leaf functions with simple control flow and limited dependencies, the generated tests achieve 100% line and branch coverage in our experiments. This high coverage ensures that diverse execution paths are exercised and increases the likelihood that behavioral divergences, if present, are exposed, thereby strengthening confidence in the equivalence of the tested functions.

External Validity. A potential threat is that our results may not generalize to other models or datasets. To mitigate this threat, we evaluate four representative LLMs on 385 labeled sensitive functions spanning multiple categories. This evaluation across diverse models and a nontrivial number of functions helps reduce model-specific or dataset-specific bias. Nevertheless, our study does not cover all possible models or application scenarios. We therefore encourage future work to extend this evaluation to additional models, datasets, and domains.

Construct Validity. One potential threat arises from the operationalization of "sensitive functions". We acknowledge that the identification of sensitive functions may be subject to bias or omission. To mitigate this threat, we provide explicit labeling criteria and guidelines. As described in Section II-C, we limit the scope of sensitive functions to those well-defined categories. This restricted scope reduces subjectivity by grounding the construct in established literature. All functions are labeled according to these predefined criteria, and the process is conducted independently by multiple authors with relevant expertise, with disagreements resolved through discussion. These measures improve the objectivity and reproducibility of the labeling process.

An additional threat to construct validity lies in our dataset construction process. Due to the large number of functions labeled (10,984 functions), the process is rather adhoc. Thus, to ensure the quality of the dataset, we performed a second round of labeling with statistically significant, randomly selected samples (372, 95% confidence level with a 5% margin of error). This second round of labeling followed a more rigorous protocol. As described in Section II-C, we restricted the definition of sensitive functions to well-established categories informed by prior TEE-related studies [24], [33], including *Encryption*, *Decryption*, *Signature*, *Verification*, *Hash*, *Seed Generation*, *Random Number Generation*, *Serialization*, and *Deserialization*. For each function, we applied the following ordered guidelines to determine sensitivity: (1) checking for explicit comments indicating the presence of sensitive operations; (2) identifying invocations of known cryptographic or deserialization APIs (e.g., `hashlib. sha256, cryptography. fernet.Fernet`); and (3) determining whether the function implements a complete sensitive logic pattern, such as AES-based encryption. We employed a quality-control process in which two authors independently reviewed each sample, while a third author adjudicated any disagreements. Each function in the sampled subset was independently reviewed by two authors. The inter-rater agreement reached an "almost perfect" level [73], with a Cohen's κ of 0.907. The remaining 11 disagreements were resolved through adjudication by a third author. We find that the labeling outcomes for the sampled subset are consistent with the initial labeling results in 97.5% of cases, providing strong evidence for the reliability of the dataset.

B. Limitation

AUTOTEE currently hardcodes environment variables in the transformed code. This approach may not be suitable for migrating our generated code across different hardware platforms. To deal with this issue, humans can manually check those hardcoded environment variables and modify them. Our goal is to automate some steps but not to fully exclude humans in the loop.

For experiment, our dataset is primarily composed of cryptographic projects, which may suggest limited diversity in software domains. We intentionally selected these projects to ensure a sufficient number of sensitive functions, which serve as positive samples in our experiments. Although these projects center on cryptography, they have diverse application contexts like Android key management applications, and cloud service plugins. We acknowledge that, in many real-world workloads, the proportion of sensitive functions is typically lower than in our dataset. However, the methodology of AUTOTEE is domain-agnostic and is not restricted to cryptographic projects. As a result, the techniques proposed in AUTOTEE remain applicable to conventional software projects. In future work, we plan to broaden our evaluation to include a wider range of projects to further assess generalizability.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

VII. Related Work

A. Application Porting to TEE

Lejacon [74] developed a customized JVM approach to run Java applications on SGX, isolating them within a secure environment to protect the confidentiality of code and data. While successful, this method increases the Trusted Computing Base (TCB) size, expanding the attack surface and potential security risks. TEESlice [1] enhances the security of on-device machine learning by accurately isolating privacy-sensitive weights in models. It employs a partition-before-training strategy, dividing models into backbone and private slices, and uses a dynamic pruning algorithm to optimize slice size, ensuring that they fit within TEEs without loss of accuracy. MyTEE [5] provides TEE functionality for embedded devices lacking full TrustZone support. It utilizes stage-2 page tables for TEE isolation from the untrusted OS and implements a DMA filter to prevent malicious memory access. Secure I/O is achieved by delegating peripheral requests to the untrusted OS, while essential components are protected by paging. For safeguarding confidential environments, ACAI [75] utilizes Arm's Confidential Computing Architecture (CCA) to securely integrate accelerators like GPUs and FPGAs. This method extends CCA's security features to these devices, ensuring isolation at the virtual machine level and protection against both software and physical threats. To address the excessively large Trusted Computing Base in TEE application adaptation, Lind et al. [17] propose a partitioning method that extracts security-sensitive components and places them within a secure enclave, ensuring that the enclave's code does not compromise data integrity or confidentiality.

In contrast, we migrate only a small portion of critical code into the TEE as native code. This approach allows AutoTEE to maintain functionality while avoiding the introduction of additional runtime overhead.

B. Code Transformation

Hong et al. [76] propose enhancing C-to-Rust translation through the automatic replacement of unsafe features with safe alternatives. While Rust's ownership type system prevents memory and thread bugs, current translators only perform syntactic conversions, requiring developers to perform manual refactoring. Their approach focuses on replacing unsafe features, such as the lock API and output parameters. However, their approach is limited to C transformations. CodeStylist [77] utilizes neural methods for code style transfer, building upon existing code language models pre-trained on extensive open-source codebases and fine-tuned through a multi-task training approach to handle diverse style transformations. However, it requires a substantial amount of pre-training data. BabelTower [78] presents a learning-based framework that automates the transformation of sequential C code into parallel CUDA code. It effectively addresses GPU programming complexities by employing an extensive dataset of compute-intensive functions and utilizing back-translation, combined with a discriminative reranker, to handle unpaired corpora and semantic transformations. hmCodeTrans [79] introduces a method for interactive human-machine collaboration in code translation. The approach defines two collaboration patterns: prefix-based and segment-based, which allow software engineers' edits to guide the model toward improved retranslation. Furthermore, response time is reduced through an attention cache module that avoids redundant prefix inference and a suffix splicing module that minimizes unnecessary suffix computation.

We utilize an LLM to assist in code transformation, while simultaneously incorporating compilability checks and equivalence validation to ensure the functional equivalence of the transformation. This process does not require prior training data.

VIII. Conclusion and Future Work

Despite the strong security protections offered by TEEs, adapting existing programs to leverage these guarantees remains challenging due to the need for extensive domain knowledge and manual intervention. The limited resources of TEEs and the complexity of identifying and transforming security-sensitive functions have hindered their broader adoption. To address these challenges, we propose AutoTEE, an automated LLM-enabled approach. AutoTEE automatically identifies, partitions, and transforms sensitive functions in high-level language programs (e.g., Java and Python), porting them to TEEs while ensuring compatibility with secure environments like Intel SGX through secure communication channels. Specifically, AutoTEE incorporates robust iterative transformation and validation mechanisms, utilizing compiler feedback for compilability and functional equivalence checks to guarantee the executability and functional consistency of the TEE-adapted code. Our evaluation, when applying GPT-4o for transformation, AutoTEE achieved high success rates in porting, with 91.8% of Java code and 84.3% of Python code successfully transformed while preserving functional equivalence and successful compilation.

AutoTEE is designed with modularity and flexibility in mind, making it adaptable to other types of sensitive code and to additional programming languages. Our LLM-driven approach enables source-level analysis and transformation without reliance on specific runtime environments, thus supporting portability across languages provided appropriate parsing and code analysis tools are available. AutoTEE can be readily extended to sensitive operations, such as network communication, privilege management, or resource-intensive workflows, with recognizable code patterns.

Currently, the migration of leaf functions into the TEE environment is limited to basic external library calls that have direct equivalents in the target TEE. In future work, we will broaden this support to cover more intricate invocation scenarios, such as those involving complex data structures and non-standard external libraries. To strengthen functional equivalence verification, we plan to adopt a more rigorous validation regime that incorporates comprehensive test suites going beyond basic

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

input-based tests, thereby improving the transformation success rate.

The observed integration failures also highlight an area for improvement. In our current study, we only use LLM to generate test inputs. In the future, we plan to (i) use LLM to generate integration and system tests, and (ii) use the outcomes of these tests as feedback for AUTOTEE to refine the transformed program. We also plan to develop a solution that can proactively identify functions whose transformation can cause integration errors and exclude those.

ACKNOWLEDGEMENT

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore, and the Cyber Security Agency of Singapore.

REFERENCES

[1] Z. Zhang et al., "No privacy left outside: On the (in-) security of TEE-shielded DNN partition for on-device ml," in *Proc. IEEE Symp. Secur. Privacy (SP)*. Piscataway, NJ, USA: IEEE Press, 2024, pp. 3327–3345.

[2] C. Liu et al., "Extending on-chain trust to off-chain–trustworthy blockchain data collection using trusted execution environment (TEE)," *IEEE Trans. Comput.*, vol. 71, no. 12, pp. 3268–3280, Dec. 2022.

[3] R. Zhang, N. Zhang, A. Moini, W. Lou, and Y. T. Hou, "Privacyscope: Automatic analysis of private data leakage in TEE-protected applications," in *Proc. IEEE 40th Int. Conf. Distrib. Comput. Syst. (ICDCS)*. Piscataway, NJ, USA: IEEE Press, 2020, pp. 34–44.

[4] W. Wang, W. Liu, H. Chen, X. Wang, H. Tian, and D. Lin, "Trust beyond border: Lightweight, verifiable user isolation for protecting in-enclave services," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 522–538, Jan./Feb. 2023.

[5] S. Han and J. Jang, "MyTEE: Own the trusted execution environment on embedded devices," in *Proc. 30th Annu. Netw. Distrib. System Secur. Symp. (NDSS)*. Reston, VA, USA: The Internet Soc., 2023.

[6] C. Ma et al., "BiTDB: Constructing a built-in TEE secure database for embedded systems," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4472–4485, Sep. 2024.

[7] J. Wang, Y. Du, X. Wang, C. Ma, D. Lu, and N. Xi, "TEE-assisted time-scale database management system on IOT devices," in *Proc. 13th Int. Conf. Internet Things*, 2023, pp. 253–259.

[8] A. Ait Messaoud, S. Ben Mokhtar, and A. Simonet-Boulogne, "TEE-based key-value stores: A survey," *VLDB J.*, vol. 34, no. 1, pp. 1–22, 2025.

[9] J. Ahn et al., "DISKSHIELD: A data tamper-resistant storage for intel SGX," in *Proc. 15th ACM Asia Conf. Computer Commun. Secur.*, 2020, pp. 799–812.

[10] L. Zhao and M. Mannan, "TEE-aided write protection against privileged data tampering," in *Proc. 26th Annu. Netw. Distrib. System Secur. Symp. (NDSS)*. Reston, VA, USA: The Internet Soc., 2019.

[11] C. Stathakopoulou, S. Rüsch, M. Brandenburger, and M. Vukolic, "Adding fairness to order: Preventing front-running attacks in BFT protocols using TEEs," in *Proc. 40th Int. Symp. Reliable Distrib. Syst. (SRDS)*. Piscataway, NJ, USA: IEEE Press, 2021, pp. 34–45.

[12] M. Zhang, Q. Zhang, S. Zhao, Z. Shi, and Y. Guan, "SoftME: A software-based memory protection approach for TEE system to resist physical attacks," *Secur. Commun. Netw.*, vol. 2019, no. 1, 2019, Art. no. 8690853.

[13] Intel Corporation, "Intel software guard extensions (SGX)," Accessed: Sep. 2025. [Online]. Available: https://www.intel.com/content/www/us/en/architecture-and-technology/software-guard-extensions.html

[14] Intel Corporation, "Intel trusted domain extensions (TDX)," Accessed: Sep. 2025. [Online]. Available: https://www.intel.com/content/www/us/en/architecture-and-technology/trusted-domain-extensions.html

[15] AMD, "Secure encrypted virtualization (SEV)," Accessed: Sep. 2025. [Online]. Available: https://www.amd.com/en/technologies/sev

[16] Arm Holdings, "Arm trustzone technology," Accessed: Sep. 2025. [Online]. Available: https://www.arm.com/architecture/security-architecture/trustzone

[17] J. Lind et al., "Glamdring: Automatic application partitioning for intel {SGX}," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC 17)*, 2017, pp. 285–298.

[18] M. Wu et al., "Toward an SGX-friendly java runtime," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 44–57, Jan. 2024.

[19] X. Miao et al., "Lejacon: A lightweight and efficient approach to java confidential computing on SGX," in *Proc. 45th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*. Piscataway, NJ, USA: IEEE Press, 2023, pp. 1648–1660.

[20] J. Qian, L. Wang, and X. Zhou, "A lightweight approach to detect memory leaks in javascript (S)," in *Proc. 30th Int. Conf. Softw. Eng. Knowl. Eng., Hotel Pullman*, *Redwood City*, *California, USA*. Skokie, Illinois, USA: KSI Res. Inc. and Knowl. Syst. Inst. Graduate School, 2018, pp. 581–582.

[21] A. Shahoor, A. Y. Khamit, J. Yi, and D. Kim, "LeakPair: Proactive repairing of memory leaks in single page web applications," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1175–1187.

[22] J. Seo et al., "SGX-shield: Enabling address space layout randomization for SGX programs," in *Proc. 24th Annu. Netw. Distrib. System Secur. Symp. (NDSS)*. Reston, VA, USA: The Internet Soc., 2017.

[23] K. Shanker, A. Joseph, and V. Ganapathy, "An evaluation of methods to port legacy code to SGX enclaves," in *Proc. 28th ACM Joint Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng. (ESEC/FSE)*. New York, NY, USA: ACM, 2020, pp. 1077–1088.

[24] P. Wang et al., "Building and maintaining a third-party library supply chain for productive and secure SGX enclave development," in *Proc. 42nd IEEE/ACM Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2020, pp. 100–109.

[25] S. Yao et al., "React: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learn. Representat. (ICLR)*. OpenReview.net, 2023.

[26] OpenTEE Project, "OpenTEE: Open trusted execution environment," Accessed: Sep. 2025. [Online]. Available: https://opentee.org

[27] D. Yan, Z. Gao, and Z. Liu, "A closer look at different difficulty levels code generation abilities of chatGPT," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*. Piscataway, NJ, USA: IEEE Press, 2023, pp. 1887–1898.

[28] Z. Zeng, H. Tan, H. Zhang, J. Li, Y. Zhang, and L. Zhang, "An extensive study on pre-trained models for program understanding and generation," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Test. Anal. (ISSTA)*, 2022, pp. 39–51.

[29] A. Mastropaolo, E. Aghajani, L. Pascarella, and G. Bavota, "An empirical study on code comment completion," in *Proc. IEEE Int. Conf. Softw. Mainten. Evol. (ICSME)*. Piscataway, NJ, USA: IEEE Press, 2021, pp. 159–170.

[30] M. Geng et al., "Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, 2024, pp. 1–13.

[31] Y. Wu et al., "How effective are neural networks for fixing security vulnerabilities," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal. (ISSTA)*, 2023, pp. 1282–1294.

[32] X. Ren, X. Ye, D. Zhao, Z. Xing, and X. Yang, "From misuse to mastery: Enhancing code generation with knowledge-driven ai chaining," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*. Piscataway, NJ, USA: IEEE Press, 2023, pp. 976–987.

[33] A. Paju, M. O. Javed, J. Nurmi, J. Savimäki, B. McGillion, and B. B. Brumley, "SoK: A systematic review of TEE usage for developing trusted applications," in *Proc. 18th Int. Conf. Avail., Rel. Secur. (ARES)*, 2023, pp. 1–15.

[34] R. Han et al., "A credential usage study: Flow-aware leakage detection in open-source projects," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 722–734, 2024.

[35] R. Feng, Z. Yan, S. Peng, and Y. Zhang, "Automated detection of password leakage from public GitHub repositories," in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*, 2022, pp. 175–186.

[36] S. Dziembowski and S. Faust, "Leakage-resilient cryptography from the inner-product extractor," in *Proc. 17th Int. Conf. Theory Appl. Cryptol. Inf. Secur. (ASIACRYPT)*, D. H. Lee and X. Wang, Eds., vol. 7073, Berlin, Heidelberg: Springer, 2011, pp. 702–721.

[37] K. Arikan et al., "TEE-shirt: Scalable leakage-free cache hierarchies for TEEs," in *Proc. 31st Annu. Netw. Distrib. System Secur. Symp. (NDSS)*. Reston, VA, USA: The Internet Soc., 2024.

[38] A. M. Alam and K. Chen, "TEE-MR: Developer-friendly data oblivious programming for trusted execution environments," *Comput. Secur.*, vol. 148, 2025, Art. no. 104119.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.

[39] B. Li, F. Zhou, Q. Wang, J. Xu, and D. Feng, "SEDCPT: A secure and efficient dynamic searchable encryption scheme with cluster padding assisted by TEE," *J. Syst. Archit.*, vol. 154, 2024, Art. no. 103221.
[40] Y. Niu et al., "What you trust is insecure: Demystifying how developers (mis)use trusted execution environments in practice," 2026, *arxiv:2512.17363*.
[41] Q. Chen et al., "Countermeasures against information leakage induced by data serialization effects in a RISC CPU," in *Proc. 4th Int. Conf. Comput. Sci. Appl. Eng.*, 2020, pp. 1–7.
[42] D. Zoni, A. Barenghi, G. Pelosi, and W. Fornaciari, "A comprehensive side-channel information leakage analysis of an in-order RISC CPU microarchitecture," *ACM Trans. Des. Automat. Electron. Syst. (TODAES)*, vol. 23, no. 5, pp. 1–30, 2018.
[43] P. Graux, J.-F. Lalande, V. V. T. Tong, and P. Wilke, "Preventing serialization vulnerabilities through transient field detection," in *Proc. 36th Annu. ACM Symp. Appl. Comput.*, 2021, pp. 1598–1606.
[44] X. Chen et al., "Tabby: Automated gadget chain detection for java deserialization vulnerabilities," in *Proc. 53rd Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*. Piscataway, NJ, USA: IEEE Press, 2023, pp. 179–192.
[45] I. Sayar, A. Bartel, E. Bodden, and Y. Le Traon, "An in-depth study of java deserialization remote-code execution exploits and vulnerabilities," *ACM Trans. Softw. Eng. Method.*, vol. 32, no. 1, pp. 1–45, 2023.
[46] Y. Song, Y. Geng, J. Wang, S. Gao, and W. Shi, "Permission sensitivity-based malicious application detection for android," *Secur. Commun. Netw.*, vol. 2021, no. 1, 2021, Art. no. 6689486.
[47] M. Cen, F. Jiang, and R. Doss, "RansoGuard: A RNN-based framework leveraging pre-attack sensitive APIs for early ransomware detection," *Comput. Secur.*, vol. 150, 2025, Art. no. 104293.
[48] Y. Liu, S. Dhar, and E. Tilevich, "Only pay for what you need: Detecting and removing unnecessary TEE-based code," *J. Syst. Softw*, vol. 188, 2022, Art. no. 111253.
[49] B. Roziere, M.-A. Lachaux, L. Chanussot, and G. Lample, "Unsupervised translation of programming languages," in *Advances in Neural Information Processing System*, vol. 33, pp. 20601–20611, 2020.
[50] S. Chakraborty, Y. Ding, M. Allamanis, and B. Ray, "CODIT: Code editing with tree-based neural models," *IEEE Trans. Softw. Eng.*, vol. 48, no. 4, pp. 1385–1399, Apr. 2022.
[51] S. Kulal et al., "SpoC: Search-based pseudocode to code," in *Advances in Neural Information Processing System*, vol. 32, 2019.
[52] G. Ryan et al., "Code-aware prompting: A study of coverage-guided test generation in regression setting using LLM," in *Proc. ACM Softw. Eng.*, 2024, vol. 1, no. FSE, pp. 951–971.
[53] S. Alagarsamy, C. Tantithamthavorn, and A. Aleti, "A3test: Assertion-augmented automated test case generation," *Inf. Softw. Technol.*, vol. 176, 2024, Art. no. 107565.
[54] H. Wang et al., "Towards memory safe enclave programming with rust-SGX," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 2333–2350.
[55] S. Wan, M. Sun, K. Sun, N. Zhang, and X. He, "RusTEE: Developing memory-safe ARM trustzone applications," in *Proc. 36th Annu. Comput. Secur. Appl. Conf. (ACSAC)*. New York, NY, USA: ACM, 2020, pp. 442–453.
[56] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Advances in Neural Information Processing System*, vol. 30, 2017.
[57] OpenAI, "Gpt-4o," 2024. [Online]. Available: https://openai.com
[58] B. Hui et al., "Qwen2.5 technical report," in *CoRR*, vol. abs/2409.12186, 2024.
[59] A. Liu et al., "Deepseek-v3 technical report," 2024, *arXiv:2412.19437*.
[60] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," Accessed: Sep. 2025. [Online]. Available: https://ai.meta.com/llama/
[61] A. Ziegler et al., "Productivity assessment of neural code completion," in *Proc. 6th ACM SIGPLAN Int. Symp. Mach. Program.*, 2022, pp. 21–29.
[62] H. Chen, D. Dean, and D. A. Wagner, "Model checking one million lines of c code," in *NDSS*, vol. 4, pp. 171–185, 2004.
[63] Y. Acar et al., "Comparing the usability of cryptographic APIs," in *Proc. IEEE Symp. Secur. Privacy (SP)*. Piscataway, NJ, USA: IEEE Press, 2017, pp. 154–171.
[64] A. S. Ami, N. Cooper, K. Kafle, K. Moran, D. Poshyvanyk, and A. Nadkarni, "Why crypto-detectors fail: A systematic evaluation of cryptographic misuse detection techniques," in *2Proc. IEEE Symp. Secur. Privacy (SP)*. Piscataway, NJ, USA: IEEE Press, 2022, pp. 614–631.
[65] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, no. 1, pp. 50–60, Mar. 1947.
[66] B. Ray, D. Posnett, V. Filkov, and P. Devanbu, "A large scale study of programming languages and code quality in GitHub," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2014, pp. 155–165.
[67] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surveys (CSUR)*, vol. 51, no. 4, pp. 1–37, 2018.
[68] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
[69] Microsoft, "Microsoft copilot is your companion to inform, entertain and inspire. get advice, feedback and straightforward answers." Accessed: Sep. 2025. [Online]. Available: https://copilot.microsoft.com/
[70] N. Merrill, "Better not to know?: The SHA1 collision & the limits of polemic computation," in *Proc. Workshop Comput. Within Limits (LIMITS)*, Santa Barbara, California, USA, B. A. Nardi and B. Tomlinson, Eds. New York, NY, USA: ACM, 2017, pp. 37–42.
[71] X. Wang, Y. L. Yin, and H. Yu, "Finding collisions in the full SHA-1," in *Proc. 25th Annu. Int. Cryptol. Conf.*, Santa Barbara, California, USA. Berlin, Heidelberg: Springer, 2005, pp. 17–36.
[72] Fortanix, "Rust SGX," Accessed: Sep. 2025. [Online]. Available: https://github.com/fortanix/rust-sgx
[73] A. J. Viera and J. M. Garrett, "Understanding interobserver agreement: The kappa statistic," *Fam. Med.*, vol. 37, no. 5, pp. 360–363, 2005.
[74] X. Miao et al., "Lejacon: A lightweight and efficient approach to java confidential computing on SGX," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*. Piscataway, NJ, USA: IEEE Press, 2023, pp. 1648–1660.
[75] S. Sridhara, A. Bertschi, B. Schlüter, M. Kuhne, F. Aliberti, and S. Shinde, "ACAI: Protecting accelerator execution with arm confidential computing architecture," in *Proc. 33rd USENIX Security Symp. (USENIX Security 24)*. Philadelphia, PA: USENIX Assoc., 2024, pp. 3423–3440.
[76] J. Hong, "Improving automatic C-to-rust translation with static analysis," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.: Companion Proc. (ICSE-Companion)*. Piscataway, NJ, USA: IEEE Press, 2023, pp. 273–277.
[77] C.-K. Ting, K. Munson, S. Wade, A. Savla, K. Kate, and K. Srinivas, "CodeStylist: A system for performing code style transfer using neural networks," in *Proc. Conf. Artif. Intell. (AAAI)*, 2023, vol. 37, no. 13, pp. 16485–16487.
[78] Y. Wen et al., "BabelTower: Learning to auto-parallelized program translation," in *Proc. Int. Conf. Mach. Learn. (ICML)*. Brookline, MA, USA: PMLR, 2022, pp. 23685–23700.
[79] J. Liu et al., "hmCodeTrans: Human-machine interactive code translation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 5, pp. 1163–1181, May 2024.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on May 15,2026 at 01:53:18 UTC from IEEE Xplore. Restrictions apply.